

OrangeOS v1.0

Ring 3核心代码定位 与答辩讲解

[用户页面](#) | [页表权限](#) | [用户现场](#) | [iretd](#) | [TSS](#) | [int 0x80](#) | [页故障](#) | [资源回收](#)

项目路径: D:\Oranges
核心执行链代码导读

阅读导航与完整链路

本手册对应答辩回答“我印象最深的是Ring 3用户态、分页保护与系统调用的完整连接”。现场看代码时，按下列顺序打开文件。

顺序	代码位置	职责
1	kernel/usermode32.asm:42	分配并映射用户三页
2	kernel/paging32.asm:87	建立带User权限的PDE/PTE
3	kernel/process32.asm:304	构造PID4的52 B用户现场
4	kernel/usermode32.asm:274	同步路径通过iretd进入Ring3
5	kernel/gdt32.asm:16	建立TSS的SS0/ESP0
6	kernel/idt32.asm:41	安装DPL3的int 0x80门
7	kernel/syscall32.asm:30	调用号分派和用户指针校验
8	kernel/idt32.asm:115	区分用户/内核页故障
9	kernel/process32.asm:392	FAULTED/EXITED与资源回收

完整执行链

OrangeFS读取OEX2 → 分配物理页 → 建立用户映射 → 构造PID4现场 → 调度器切换ESP → iretd进入Ring3 → int 0x80 → TSS换到Ring0栈 → 校验并执行 → exit或#PF → 释放三页 → 调度下一任务

一句话总述

usermode_prepare32准备用户地址空间，process_spawn_user32构造可由popad/iretd恢复的现场，IDT和TSS提供受控跨级入口，isr_syscall32验证并执行服务，页故障或退出最后进入公共终止路径回收资源。

1 用户代码、数据和栈页面

文件：kernel/usermode32.asm；函数：usermode_prepare32；起始行：42。
它连续申请三个4 KiB物理页，并分别映射到固定用户虚拟地址。分配物理页和建立虚拟映射是两个不同动作。

用户区域	虚拟地址	物理页来源
代码页	0x40000000	memory_alloc_page32
数据页	0x40001000	memory_alloc_page32
栈页	0x40002000；栈顶0x40003000	memory_alloc_page32

```
call memory_alloc_page32
mov [user_code_phys],eax
mov ebx,eax
mov eax,USER_CODE
call paging_map_user32

call memory_alloc_page32
mov [user_data_phys],eax
mov eax,USER_DATA
call paging_map_user32

call memory_alloc_page32
mov [user_stack_phys],eax
mov eax,USER_STACK_TOP-4096
call paging_map_user32
```

失败路径会调用usermode_release32清理已经成功的部分，避免准备过程半途失败后持续占用页面。

现场讲解词

这里不是直接把用户代码放进任意内存，而是先从物理页分配器取得三个4 KiB页面，再分别映射到固定的用户代码、数据和栈地址。用户任务运行时使用页数增加3，退出或故障后应回到原值。

释放代码：kernel/usermode32.asm:100

```
mov eax,USER_CODE
call paging_unmap_user32
mov eax,[user_code_phys]
call memory_free_page32
; data stack
```

2 用户页表权限与TLB

文件: kernel/paging32.asm; 函数: paging_map_user32; 起始行: 87。

32位线性地址按10位页目录索引、10位页表索引和12位页内偏移拆分。

虚拟地址[31:22] → PDE索引; [21:12] → PTE索引; [11:0] → 4 KiB页内偏移

```
mov edx,esi
shr edx,22      ; page directory index
...
mov eax,esi
shr eax,12
and eax,0x3FF   ; page table index
```

PDE与PTE都设置0x07: Present、Writable、User。任意一级没有User位, Ring 3访问都会被拒绝。

```
or eax,0x07
mov [ebx+edx*4],eax    ; PDE
...
or edi,0x07
mov [ecx+eax*4],edi    ; PTE
invlpg [esi]
```

invlpg使该虚拟页的旧TLB缓存失效, 否则CPU可能继续使用修改前的地址转换。解除映射位于paging_unmap_user32 (148行), 它清零PTE后同样执行invlpg。

现场讲解词

这一段体现分页同时完成地址转换和保护。物理页地址写入PTE, 低位0x07赋予用户读写权限。映射变化后必须刷新TLB, 保证CPU不再使用旧的缓存映射。

3 构造可调度的Ring 3现场

文件：kernel/process32.asm；函数：process_spawn_user32；起始行：304。

PID4的现场位于专用内核栈。代码先清空13个dword，即52 B，再填入中断恢复所需字段。

偏移	字段	值/含义
0-31	pushad通用寄存器现场	8个dword
+32	EIP	用户入口，通常0x40000000
+36	CS	0x1B，RPL=3
+40	EFLAGS	0x202，IF=1
+44	用户ESP	0x40003000
+48	用户SS	0x23，RPL=3

```
mov edi,USER_KERNEL_FRAME_TOP-52
xor eax,eax
mov ecx,13
rep stosd
...
mov [edi+32],eax
mov dword [edi+36],0x1B
mov dword [edi+40],0x00000202
mov dword [edi+44],USER_STACK_TOP
mov dword [edi+48],0x23
mov [PCB_ESP],USER_KERNEL_FRAME_TOP-52
mov [PCB_STATE],PROCESS_STATE_READY
```

用户帧比同特权级内核中断帧多8 B，因为跨级iretd还要恢复用户ESP和SS。调度器不必逐项复制寄存器，只需切换到PCB保存的ESP，再popad和iretd。

现场讲解词

用户态不是修改一个变量，而是提前伪造一份CPU能够合法恢复的跨特权级现场。选择子低两位为3，EFLAGS打开中断，ESP指向用户栈顶；任务被调度后，iretd读取这份现场并真正把CPL切换为3。

4 iretd降权与TSS换栈

同步user命令的直接降权代码位于kernel/usermode32.asm:274。

```
mov [kernel_saved_esp],esp
push dword USER_DS
push dword USER_STACK_TOP
pushfd
or dword [esp],0x200
push dword USER_CS
push dword USER_CODE
iretd
```

按CPU要求从后向前压入SS、ESP、EFLAGS、CS、EIP。iretd按相反方向弹出，并根据目标CS的RPL=3完成Ring0到Ring3切换。

TSS代码位于kernel/gdt32.asm:16

```
mov dword [tss32+4],TSS_ESP0 ; ESP0
mov word [tss32+8],KERNEL_DATA ; SS0
mov word [tss32+102],104 ; I/O
...
mov ax,TSS_SELECTOR
ltr ax
```

TSS在本项目中不负责软件调度。它的核心作用是当Ring3发生中断、异常或系统调用时，向CPU提供可信的Ring0栈SS0/ESP0。

现场讲解词

进入用户态靠iretd恢复五个跨级字段；从用户态重新进入内核时靠TSS换栈。两者方向相反，共同完成安全的特权级往返。

5 int 0x80受控入口

系统调用门安装于kernel/idt32.asm:41。

```
mov ebx,0x80
mov eax,isr_syscall32
call set_idt_gate32
mov byte [idt_table+0x80*8+5],11101110b
```

门属性0xEE表示Present=1、DPL=3、32位中断门。DPL=3是Ring3主动执行int 0x80的必要条件；其他内核门通常DPL=0。

调用号分派位于kernel/syscall32.asm:30

EAX	系统调用
0	get_ticks
1	get_free_pages
2	get_pid
3	exit
4	set_result
5	write

```
cmp eax,SYS_GET_TICKS
je .ticks
...
cmp eax,SYS_WRITE
je .write
mov eax,0xFFFFFFFF
```

SYS_WRITE验证来源CS、长度上限127、指针加法溢出，以及整个缓冲区是否落在合法用户页范围；随后复制到128 B内核缓冲区再输出。

```
mov eax,[esp+28]
and eax,3
cmp eax,3
jne .write_invalid
cmp ecx,SYS_WRITE_MAX
ja .write_invalid
mov edx,esi
add edx,ecx
jc .write_invalid
...
mov edi,syscall_write_buffer
rep movsb
```

现场讲解词

系统调用门只解决用户是否允许进入内核，真正的安全边界还包括调用号白名单和参数校验。内核不能直接信任用户指针，所以先验证完整范围，再复制到自己的缓冲区。

6 页故障隔离和资源回收

页故障入口位于kernel/idt32.asm:115。旧CS的低两位用于判断故障来源。

```
pushad
mov eax,[esp+40]
and eax,3
cmp eax,3
jne .kernel_fault
mov eax,cr2      ; fault address
mov ebx,[esp+32]  ; error code
mov ecx,[esp+36]  ; faulting EIP
call process_fault_current32
mov esp,eax
popad
iretd
```

Ring3故障进入process_fault_current32；Ring0故障则显示CR2并hlt停机，因为继续运行不再安全。

公共终止路径位于kernel/process32.asm:392

```
process_fault_current32:
    mov [process_fault_address],eax
    mov [process_fault_error],ebx
    mov [process_fault_eip],ecx
    mov eax,PROCESS_STATE_FAULTED

process_terminate_current32:
    mov [PCB_STATE],eax
    call usermode_release32
    ; round-robin find READY
    mov eax,[next_PCB_ESP]
    ret
```

返回值不是普通状态码，而是下一任务的内核帧ESP。页故障入口直接切换ESP并iretd，因此不会回到故障指令。正常SYS_EXIT也进入同一公共终止思路，只是状态为EXITED。

现场讲解词

分页先阻止非法访问，页故障入口再根据旧CS确认它来自用户态，只终止PID4并回收三页，然后恢复其他任务。这就是用户程序错误不会拖垮整个内核的代码依据。

7 现场展示顺序与高频追问

三分钟代码展示

时间	打开位置	要说的话
0:00-0:35	usermode32.asm:42	三页物理分配与固定虚拟映射
0:35-1:10	process32.asm:304	52 B跨级现场与PID4 READY
1:10-1:35	gdt32.asm:16	TSS提供可信Ring0栈
1:35-2:05	idt32.asm:41 + syscall32.asm:30	DPL3入口、分派和指针校验
2:05-2:40	idt32.asm:115	CR2、错误码、旧CS来源判断
2:40-3:00	process32.asm:392	FAULTED/EXITED、释放三页、切下一任务

高频追问

- 为什么不是改变量就进入Ring3？CPU特权级由CS/CPL、门描述符和分页权限共同决定。
- 为什么用户帧是52 B？pushad 32 B，加EIP/CS/EFLAGS 12 B，再加跨级ESP/SS 8 B。
- 为什么PDE和PTE都要User？x86逐级检查权限，任何一级限制都会拒绝Ring3。
- TSS是否调度任务？不负责；软件调度保存PCB/ESP，TSS只提供跨级SS0/ESP0。
- 为什么SYS_WRITE要复制？用户地址不可信，先验证再copy-from-user可以隔离内核输出路径。
- 为什么Ring0页故障停机？内核关键状态可能损坏，继续运行风险大于恢复价值。
- 怎么证明无泄漏？自动测试检查用户任务前N、运行中N+3、结束后N。

30秒总结

我最印象深刻的是Ring3、分页和系统调用的连接。内核先分配三个用户物理页并建立User映射，再构造包含EIP、CS、EFLAGS、ESP、SS的52字节现场，通过调度和iretd进入Ring3。用户只能通过DPL3的int 0x80门进入内核，CPU依据TSS切到可信栈，内核验证参数后执行。非法访问触发#PF，只把用户任务标记FAULTED、释放三页并切换其他任务，从而形成权限、服务和故障隔离闭环。

代码位置速记

准备页 usermode32:42 → 映射 paging32:87 → 构帧 process32:304 → 降权 usermode32:274 → TSS gdt32:16 → 门 idt32:41 → syscall syscall32:30 → #PF idt32:115 → 回收 process32:392